APM — 100 questions de cours / 100 道课程问答题

Préparation du DST du 22 octobre 2026 / 2026 年 10 月 22 日笔试复习

Questions d'entraînement originales, construites à partir des notions abordées en CM et des TD1, TD2 et TP1. La priorité suit les contenus effectivement étudiés. Les bases des API CUDA et quelques notions OpenMP complètent l'ensemble ; NCCL et l'implémentation complète d'un GEMM optimisé sont hors de cette série. Il ne s'agit pas d'un sujet officiel ni d'une prédiction du sujet.

这是一套依据课程与 TD1、TD2、TP1 整理的自编练习，优先围绕已经学习的内容，补充 CUDA API 基础和少量 OpenMP 概念。本套不涉及 NCCL 或完整的 GEMM 优化实现，也不要求编写完整程序。题目与答案均为复习用参考，不是老师公布的考试题或官方答案。

Pour chaque question, formulez d'abord une réponse de deux à quatre phrases en français, puis comparez-la à la réponse proposée. Une formulation équivalente et techniquement correcte convient ; il n'est pas nécessaire de réciter mot à mot.

每题先尝试用两到四句法语回答，再对照参考答案。抓住定义、条件和因果关系即可，不必逐字背诵。每道法语问题后紧跟中文译文，答案也按法中顺序排列。

Les numéros tels que 2.4 ou 3.5 désignent les sections du **guide de révision**, et non les numéros des diapositives. Les liens « Repères » mènent à la version chinoise ; la version française conserve les mêmes numéros.

文中"2.4""3.5"等编号指 **复习讲义的小节**，不是课件页码。"复习讲义"链接指向中文版；法语版使用相同编号。

Guide français / 法语复习讲义 · 中文复习讲义 · 100 QCM / 100 道选择题 · 索引

Parcours prioritaire / 优先复习顺序

Les 60 questions marquées **Prioritaire / 核心** portent sur les notions à consolider d'abord. Les 40 autres complètent la révision ; ce classement n'est pas une probabilité de présence à l'examen.

先做标为 **Prioritaire / 核心** 的 60 道题，剩余 40 道用于补齐细节。这里的"核心"表示复习顺序，不表示一定会考。

Groupe / 分组	Questions prioritaires / 核心题号
1–25 · Exécution, fonctions et indices / 执行模型、函数属性与索引	Q001、Q002、Q004、Q005、Q007、Q008、Q009、Q010、Q011、Q012、Q013、Q014、Q015、Q020、Q021
26–50 · Mémoire et accès aux données / 内存与数据访问	Q026、Q028、Q029、Q030、Q031、Q032、Q033、Q034、Q035、Q036、Q038、Q039、Q040、Q041、Q050
51–75 · Asynchronisme et synchronisation / 异步执行与同步	Q051、Q052、Q053、Q055、Q057、Q058、Q059、Q060、Q061、Q063、Q064、Q068、Q069、Q071、Q072
76–100 · TD, TP et bases des API / TD、TP 与 API 基础	Q076、Q078、Q080、Q081、Q082、Q083、Q084、Q085、Q086、Q087、Q088、Q089、Q091、Q092、Q095

Exécution, fonctions et indices / 执行模型、函数属性与索引

Q001

Prioritaire / 核心

Qu'est-ce qu'un accélérateur de calcul, et pourquoi ne remplace-t-il pas nécessairement le CPU ?

什么是计算加速器？为什么它不一定取代 CPU?

Réponse proposée / 参考答案

Un accélérateur est une ressource conçue pour favoriser certaines classes de traitements. Il consacre davantage de matériel aux opérations adaptées à ces traitements, avec une moindre généralité. Le CPU reste utile pour piloter le programme et gérer les tâches de contrôle ou peu parallèles.

加速器是针对某些类型的处理任务设计的计算资源。它把更多硬件资源用于擅长的操作，同时牺牲一部分通用性。CPU 仍适合组织程序，处理复杂控制或并行性较低的任务。

Repères / 复习讲义：1.1。

Q002

Prioritaire / 核心

Pourquoi une opération identique sur un grand nombre d'éléments indépendants se prête-t-elle au calcul GPU ?

为什么对大量独立元素执行相同操作适合交给 GPU?

Réponse proposée / 参考答案

Les éléments indépendants fournissent de nombreuses tâches qui peuvent être réparties entre les threads. Une opération régulière permet à ces threads de suivre un déroulement similaire. Cela correspond à l'objectif de débit élevé du GPU, sous réserve d'une organisation correcte des données et des accès.

相互独立的元素提供了大量可分配给不同线程的任务。规律的操作也使这些线程可以沿着相似的执行路径推进。这符合 GPU 追求高吞吐量的设计，但仍需正确组织数据和访问。

Repères / 复习讲义：1.1、1.3。

Q003

Complément / 补充

Une boucle est parallélisable : est-ce suffisant pour décider de l'exécuter sur le GPU ? Justifiez.

一个循环可以并行化，是否就足以决定让它在 GPU 上执行？请说明理由。

Réponse proposée / 参考答案

Non, il faut aussi examiner la quantité de travail, les déplacements de données et la régularité du calcul. Un petit calcul peut coûter moins de temps sur le CPU que sa préparation et son exécution sur le GPU. Il faut évaluer le temps du traitement complet, pas seulement celui du kernel.

不够，还要考虑计算规模、数据移动和计算规律性。对于很小的任务，CPU 直接计算的时间可能短于 GPU 的准备与执行总时间。判断加速效果应看完整处理过程，不能只看 kernel。

Repères / 复习讲义：1.1。

Q004

Prioritaire / 核心

Décrivez la répartition du travail entre host et device dans les premiers programmes CUDA du cours.

请描述课程最初 CUDA 程序中 host 与 device 的分工。

Réponse proposée / 参考答案

Le host, exécuté sur le CPU, prépare les données et organise les ressources ainsi que les lancements. Le device exécute les instances du kernel sur le GPU. Le host récupère ensuite les résultats lorsque le calcul nécessaire est terminé.

host 在 CPU 上执行，准备数据并组织资源与启动操作。device 在 GPU 上执行 kernel 的各个实例。所需计算完成后，host 再取回结果。

Repères / 复习讲义：1.7。

Q005

Prioritaire / 核心

Dans TD1, pourquoi faut-il distinguer `kernel<<<G, B>>>(...)` du corps de la fonction kernel ?

在 TD1 中，为什么要区分 `kernel<<<G, B>>>(...)` 这条语句与 kernel 函数体?

Réponse proposée / 参考答案

L'instruction de lancement appartient au programme hôte : elle demande au GPU d'exécuter le kernel avec une configuration donnée. Le corps de la fonction est exécuté par les threads du device. Une même définition de kernel produit ainsi de nombreuses instances, distinguées notamment par leurs indices.

启动语句属于主机程序，用于请求 GPU 按指定配置执行 kernel。函数体则由 device 上的线程执行。同一份 kernel 定义因此对应许多执行实例，各实例可以通过索引等信息区分。

Repères / 复习讲义：1.7、1.12。

Q006

Complément / 补充

Pourquoi l'affirmation « un thread CUDA correspond toujours à un cœur CUDA » est-elle incorrecte ?

为什么“一个 CUDA 线程始终对应一个 CUDA 核心”是不正确的?

Réponse proposée / 参考答案

Un thread est une instance logique du kernel, alors qu'un cœur CUDA est une unité matérielle de calcul. Les unités matérielles exécutent le travail des threads au fil de l'ordonnancement. Le nombre de threads lancés peut donc être bien supérieur au nombre de cœurs.

线程是 kernel 的逻辑执行实例，而 CUDA 核心是硬件计算单元。硬件单元通过调度在执行过程中处理不同线程的工作。因此，启动的线程数可以远大于核心数。

Repères / 复习讲义：1.3。

Q007

Prioritaire / 核心

Distinguez la grille, le bloc et le SM dans le modèle CUDA de base.

请区分 CUDA 基础模型中的网格、线程块与 SM。

Réponse proposée / 参考答案

La grille rassemble les blocs d'un lancement et chaque bloc contient des threads. La grille et le bloc sont des organisations logiques choisies par le programmeur. Le SM est une ressource matérielle du GPU qui accueille et exécute des blocs selon ses capacités.

网格包含一次启动的线程块，每个块包含线程。网格与块是程序员选择的逻辑组织方式。SM 是 GPU 的硬件资源，按自身容量驻留并执行线程块。

Repères / 复习讲义：1.3、1.8。

Q008

Prioritaire / 核心

Un lancement contient davantage de blocs que le GPU n'a de SM. Pourquoi cela peut-il être parfaitement valide ?

一次启动的线程块数量超过 GPU 的 SM 数量，为什么仍然完全可能合法?

Réponse proposée / 参考答案

Un SM peut accueillir plusieurs blocs lorsque ses ressources le permettent. De plus, tous les blocs d'une grille n'ont pas besoin d'être résidents simultanément. Les blocs restants peuvent être exécutés lorsque les ressources se libèrent.

只要资源允许，一个 SM 可以驻留多个块。此外，网格中的所有块不需要同时驻留。资源释放后，其余块可以继续执行。

Repères / 复习讲义：1.3。

Q009

Prioritaire / 核心

Comment les warps sont-ils formés dans un bloc multidimensionnel ? Que se passe-t-il si sa taille n'est pas un multiple de 32 ?

多维线程块中的 warp 如何组成？块大小不是 32 的倍数时会怎样?

Réponse proposée / 参考答案

Les threads sont regroupés selon leur numéro local linéaire, avec x variant le plus vite. Chaque warp possède 32 positions ; le dernier warp peut ne contenir qu'une partie de ces threads. Des threads de blocs différents ne sont pas réunis pour compléter ce dernier warp.

线程按块内线性编号组成 warp，其中 x 变化最快。每个 warp 有 32 个线程位置，最后一个 warp 可能只有部分位置对应实际线程。不同块的线程不会被拼在一起补满这个 warp。

Repères / 复习讲义：1.3、1.13。

Q010

Prioritaire / 核心

Que signifie le modèle SIMT du point de vue du programmeur et de l'exécution ?

从程序员和执行方式两个角度看，SIMT 模型意味着什么?

Réponse proposée / 参考答案

Le programmeur écrit le travail d'un thread, avec ses propres indices et ses propres données. Le GPU exécute les instructions de ces threads par groupes appelés warps. Il faut donc distinguer les instances logiques du programme de leur exécution groupée.

程序员描述一个线程的工作，这个线程有自己的索引和数据。GPU 以 warp 为单位成组执行这些线程的指令。因此，需要区分程序中的逻辑实例与硬件上的成组执行。

Repères / 复习讲义：1.3。

Q011

Prioritaire / 核心

Expliquez la divergence de branchement dans un warp. Pourquoi un `if` reste-t-il autorisé dans un kernel ?

解释 warp 内的分支分歧。为什么 kernel 中仍然允许使用 `if` ？

Réponse proposée / 参考答案

Il y a divergence lorsque les threads d'un même warp ne suivent pas tous le même chemin de contrôle. Les chemins concernés peuvent progresser séparément avec des threads actifs différents, ce qui peut augmenter le coût. Le branchement reste valide et peut protéger les accès ou exprimer le calcul demandé.

同一 warp 的线程没有沿着相同控制路径执行时，就出现了分歧。不同路径可能分开推进，并启用不同的线程，从而增加开销。分支仍然合法，也可能用于保护访问或表达题目所需的计算。

Repères / 复习讲义：1.3。

Q012

Prioritaire / 核心

Pourquoi la correction d'un kernel ordinaire ne doit-elle pas supposer que le bloc 0 s'exécute avant le bloc 1 ?

为什么普通 kernel 的正确性不能依赖“块 0 先于块 1 执行”这一假设?

Réponse proposée / 参考答案

La numérotation des blocs identifie leur travail mais ne fixe pas leur ordre d'exécution. Leur ordonnancement dépend des ressources disponibles et ne constitue pas une garantie donnée par la grille. Un algorithme reposant sur cet ordre implicite peut donc lire un résultat qui n'a pas encore été produit.

块编号用于标识任务，不规定执行次序。块如何被调度取决于可用资源，网格并没有给出这种顺序保证。依赖隐含顺序的算法因此可能读取尚未产生的结果。

Repères / 复习讲义：1.13。

Q013

Prioritaire / 核心

Expliquez la différence entre `gridDim`, `blockDim`, `blockIdx` et `threadIdx`.

解释 `gridDim`、`blockDim`、`blockIdx` 和 `threadIdx` 的区别。

Réponse proposée / 参考答案

`gridDim` donne le nombre de blocs par dimension et `blockDim` le nombre de threads par dimension d'un bloc. `blockIdx` donne les coordonnées du bloc courant, tandis que `threadIdx` donne celles du thread dans ce bloc. Les indices commencent à zéro et restent strictement inférieurs aux dimensions correspondantes.

`gridDim` 给出各维的块数，`blockDim` 给出一个块内各维的线程数。`blockIdx` 是当前块的坐标，`threadIdx` 是当前线程在块内的坐标。坐标从 0 开始，并严格小于对应维度的长度。

Repères / 复习讲义：1.8。

Q014

Prioritaire / 核心

Justifiez la formule `i = blockIdx.x * blockDim.x + threadIdx.x` pour une grille et des blocs 1D.

请解释一维网格和一维线程块中公式 `i = blockIdx.x * blockDim.x + threadIdx.x` 的由来。

Réponse proposée / 参考答案

Chaque bloc précédent contient `blockDim.x` threads. Le produit `blockIdx.x * blockDim.x` compte donc tous les indices attribués aux blocs précédents. L'ajout de `threadIdx.x` situe le thread dans

son propre bloc et fournit un numéro unique dans la grille.

前面的每个块都含有 `blockDim.x` 个线程。因此， `blockIdx.x * blockDim.x` 统计了前面所有块占用的编号。再加上 `threadIdx.x` ，就得到当前线程在整个网格中的唯一编号。

Repères / 复习讲义：1.9。

Q015

Prioritaire / 核心

Pourquoi utilise-t-on à la fois un nombre de blocs arrondi au supérieur et un test `i < N` dans une décomposition à un élément par thread ?

每线程处理一个元素时，为什么既要对块数向上取整，又要检查 `i < N` ？

Réponse proposée / 参考答案

L'arrondi au supérieur assure qu'il existe assez de threads pour tous les éléments. Si N n'est pas un multiple de la taille du bloc, certains threads supplémentaires n'ont pas d'élément valide. Le test `i < N` empêche ces threads d'accéder hors du tableau.

块数向上取整保证有足够的线程覆盖所有元素。当 N 不是块大小的整数倍时，多出的线程没有对应的合法元素。 `i < N` 用于阻止这些线程访问数组边界之外的位置。

Repères / 复习讲义：1.9。

Q016

Complément / 补充

Pour un tableau de N éléments indexé à partir de 0, expliquez pourquoi `i <= N` ne remplace pas correctement `i < N` .

对于一个从 0 开始编号、含 N 个元素的数组，为什么不能把 `i < N` 改成 `i <= N` ？

Réponse proposée / 参考答案

Les indices valides vont de 0 à N − 1. La condition `i <= N` autorise aussi l'indice N, qui désigne un élément au-delà du tableau. Dès qu'un thread portant cet indice existe, son accès n'est donc plus protégé.

合法下标是 0 到 N − 1。条件 `i <= N` 还会允许下标 N，而它已经位于数组之外。一旦启动配置产生这个下标的线程，该访问就不再受到正确保护。

Repères / 复习讲义：1.9。

Q017

Complément / 补充

Pour une grille 2D numérotée avec x variant le plus vite, pourquoi l'indice du bloc s'écrit-il `blockIdx.y * gridDim.x + blockIdx.x` ?

二维网格按 x 变化最快的顺序编号时，为什么块编号是 `blockIdx.y * gridDim.x + blockIdx.x`？

Réponse proposée / 参考答案

Une ligne complète de la grille contient `gridDim.x` blocs. Il faut d'abord compter les `blockIdx.y` lignes précédentes, puis avancer de `blockIdx.x` positions dans la ligne courante. La même idée s'applique aux threads d'un bloc en utilisant `threadIdx` et `blockDim`.

网格每个完整行中有 `gridDim.x` 个块。先统计前面 `blockIdx.y` 个完整行中的块，再前进到当前行内的 `blockIdx.x` 位置。把变量换成 `threadIdx` 和 `blockDim`，同样的思路也适用于块内线程。

Repères / 复习讲义：1.10。

Q018

Complément / 补充

Dans un bloc 3D où x varie le plus vite, expliquez les trois contributions à l'indice local d'un thread.

在 x 变化最快的三维线程块中，解释块内线程编号的三个组成部分。

Réponse proposée / 参考答案

Les plans z précédents contribuent `threadIdx.z * blockDim.y * blockDim.x` positions. Les lignes y précédentes du plan courant contribuent `threadIdx.y * blockDim.x`. La position dans la ligne ajoute `threadIdx.x` ; leur somme est l'indice local linéaire.

前面的 z 层贡献 `threadIdx.z * blockDim.y * blockDim.x` 个位置。当前层内前面的 y 行贡献 `threadIdx.y * blockDim.x` 个位置。再加上行内位置 `threadIdx.x`，三者之和就是块内线性编号。

Repères / 复习讲义：1.11。

Q019

Complément / 补充

Pourquoi un numéro global calculé bloc par bloc n'est-il pas toujours l'indice d'une matrice stockée par lignes ?

为什么逐块计算的全局线程编号不一定等于按行存储矩阵的数组下标?

Réponse proposée / 参考答案

La numérotation par blocs énumère tous les threads d'un bloc avant de passer au suivant. Le stockage par lignes énumère les colonnes d'une ligne entière de la matrice, même si cette ligne traverse plusieurs blocs. Pour accéder à la matrice, on calcule ses coordonnées globales puis l'indice `y * largeur + x`.

逐块编号先列出某个块内的全部线程，再进入下一个块。按行存储则先列出矩阵一整行的所有列，即使这行跨越了多个块。访问矩阵时，应先计算全局数据坐标，再使用 `y * 行宽 + x` 得到下标。

Repères / 复习讲义：1.10。

Q020

Prioritaire / 核心

Dans le scénario du cours où le host lance les kernels, comparez les fonctions `__global__` et `__device__`.

在本课程由 host 启动 kernel 的场景中，比较 `__global__` 与 `__device__` 函数。

Réponse proposée / 参考答案

Une fonction `__global__` est un kernel exécuté sur le GPU ; dans ce scénario, le host la lance avec une configuration `<<<grille, bloc>>>`. Elle retourne `void` et peut écrire ses résultats dans des buffers. Une fonction `__device__` est appelée par du code device, sans créer une nouvelle grille par cet appel ordinaire. Elle peut retourner une valeur à son appelant device selon son type de retour.

`__global__` 函数是在 GPU 上执行的 kernel；在这一场景中，host 通过 `<<<网格, 线程块>>>` 指定配置并启动它。它返回 `void`，可以把结果写入缓冲区。`__device__` 函数由设备端代码调用，这种普通调用不会新建一个网格。它可以根据返回类型，把一个值返回给设备端调用者。

Repères / 复习讲义：2.1。

Q021

Prioritaire / 核心

Que permet `__host__ __device__`, et pourquoi cela ne signifie-t-il pas que chaque appel utilise simultanément le CPU et le GPU ?

`__host__ __device__` 允许什么？为什么它不意味着每次调用都会同时使用 CPU 和 GPU?

Réponse proposée / 参考答案

Cette déclaration demande la compilation de deux versions de la fonction, l’une pour le host et l’autre pour le device. Un appel depuis le code hôte utilise la version CPU ; un appel depuis le code device utilise la version GPU. L’attribut ne lance pas lui-même de kernel et ne répartit pas automatiquement le calcul entre les deux processeurs.

这种声明要求为函数编译两个版本，分别用于 host 和 device。从主机代码调用时使用 CPU 版本，从设备端代码调用时使用 GPU 版本。该属性本身不启动 kernel，也不会自动把计算分配给两个处理器。

Repères / 复习讲义：2.1。

Q022

Complément / 补充

Pourquoi « un thread par élément » est-il une convention de décomposition et non une règle du langage CUDA ?

为什么“每线程一个元素”是任务划分约定，而不是 CUDA 语言的强制规则?

Réponse proposée / 参考答案

Un kernel peut contenir des boucles et un thread peut traiter plusieurs indices. Le programmeur choisit la répartition du travail en fonction du problème. Il doit cependant éviter de faire répéter à tous les threads la même boucle complète avec des écritures concurrentes aux mêmes endroits.

kernel 可以包含循环，一个线程也可以处理多个下标。程序员可以根据问题选择任务划分方式。不过，应避免让所有线程重复执行完整循环，并同时写入相同的位置。

Repères / 复习讲义：1.13。

Q023

Complément / 补充

Dans TD1, que permet de vérifier une émulation CPU qui parcourt séquentiellement les blocs et les threads, et que ne reproduit-elle pas ?

TD1 中通过串行遍历块与线程实现的 CPU 模拟可以检查什么？又不能复现什么?

Réponse proposée / 参考答案

Elle permet de vérifier l'indexation, la répartition des éléments et le trajet logique des données. Les boucles CPU exécutent cependant ces instances successivement. Elles ne reproduisent ni les warps ni l'ordonnancement réel et ne permettent pas de déduire les performances GPU.

这种模拟可以检查索引、元素分工和逻辑上的数据流。不过，CPU 循环依次执行这些实例。它既不复现 warp，也不复现真实调度，因此不能据此推断 GPU 性能。

Repères / 复习讲义：1.3、1.8、1.13、TD1 Q.9。

Q024

Complément / 补充

Pourquoi une intégration physique plus étroite entre CPU et GPU ne signifie-t-elle pas que toute communication est gratuite ?

为什么 CPU 与 GPU 在物理上集成得更紧密，并不意味着所有通信都没有代价?

Réponse proposée / 参考答案

L'intégration peut améliorer les interconnexions, mais les données gardent une localisation et les accès ont un coût. La proximité physique ne signifie pas automatiquement qu'une même région mémoire est partagée de la même façon. Il faut encore tenir compte du trajet des données et de la fin des calculs avant d'utiliser leurs résultats.

更紧密的集成可以改善互连，但数据仍有存放位置，访问仍有代价。物理接近并不自动意味着同一片内存以相同方式共享。使用结果时，仍要考虑数据经过的路径与相关计算是否已经结束。

Repères / 复习讲义：1.2、1.5。

Q025

Complément / 补充

Un lancement CUDA en deux dimensions impose-t-il l'utilisation d'un tableau 2D ? Expliquez le rôle des dimensions de lancement.

CUDA 的二维启动是否要求必须使用二维数组？请解释启动维度的作用。

Réponse proposée / 参考答案

Non, les dimensions de la grille et des blocs sont une organisation logique des threads. Le programmeur définit leur correspondance avec les données, qui peuvent être stockées dans un tableau linéaire. Il doit ensuite vérifier la couverture, les bornes et l'unicité des accès lorsqu'elle est requise.

不要求，网格和线程块的维度是对线程的逻辑组织方式。程序员负责定义线程坐标与数据的对应关系，数据完全可以存储在线性数组中。之后应检查覆盖范围、边界，以及需要时访问是否唯一。

Repères / 复习讲义：1.8、1.13。

Mémoire et accès aux données / 内存与数据访问

Q026

Prioritaire / 核心

Comparez `malloc`, `cudaMalloc` et `cudaMallocHost` dans le modèle mémoire du cours.

比较课程内存模型中的 `malloc`、`cudaMalloc` 和 `cudaMallocHost`。

Réponse proposée / 参考答案

`malloc` fournit de la mémoire hôte ordinaire, `cudaMalloc` un buffer device et `cudaMallocHost` de la mémoire hôte pinned. Les libérations correspondantes sont `free`, `cudaFree` et `cudaFreeHost`. L'emplacement, la possibilité d'accès et la disponibilité des données sont des questions distinctes.

`malloc` 分配普通主机内存，`cudaMalloc` 分配设备缓冲区，`cudaMallocHost` 分配主机页锁定内存。释放函数分别是 `free`、`cudaFree`、`cudaFreeHost`。分配位置、谁能访问、数据是否已准备好，需要分别判断。

Repères / 复习讲义：2.4、2.11。

Q027

Complément / 补充

Pourquoi ne faut-il pas verrouiller en pages toute la mémoire hôte disponible ?

为什么不应把所有可用主机内存都设为页锁定内存?

Réponse proposée / 参考答案

Le verrouillage empêche le système de paginer ces pages et réduit ses possibilités de gestion mémoire. Une quantité excessive peut donc pénaliser les autres usages. On réserve la mémoire pinned aux buffers pour lesquels les transferts justifient ce coût.

锁页使操作系统不能将这些页面换出，限制了系统的内存管理空间。大量锁页可能影响其他任务。应按传输需要选择 pinned 缓冲区，而不是无限扩大分配。

Repères / 复习讲义：2.11。

Q028

Prioritaire / 核心

Pourquoi passe-t-on souvent `&d_a` à `cudaMalloc`, mais `d_a` à `cudaMemcpy` ?

为什么通常向 `cudaMalloc` 传 `&d_a`，向 `cudaMemcpy` 传 `d_a`？

Réponse proposée / 参考答案

`cudaMalloc` doit modifier la variable pointeur du programme pour y écrire l'adresse obtenue ; il reçoit donc son adresse. `cudaMemcpy` travaille sur les zones de données déjà allouées et reçoit leurs pointeurs. Confondre les deux fait utiliser une mauvaise adresse.

`cudaMalloc` 要把新分配地址写回程序中的指针变量，因此需要该变量的地址。`cudaMemcpy` 操作已经分配的数据区域，接收数据指针。混淆两者会把错误的地址交给 API。

Repères / 复习讲义：2.4。

Q029

Prioritaire / 核心

Expliquez la différence entre un nombre d'éléments et une taille en octets dans les API CUDA.

解释 CUDA API 中元素个数与字节数的区别。

Réponse proposée / 参考答案

Un tableau de N éléments de type T occupe `N*sizeof(T)` octets. Les tailles de `cudaMalloc` et `cudaMemcpy` sont exprimées en octets, alors que l'addition sur un `T*` compte des éléments. On calcule donc la taille de copie et le déplacement de pointeur séparément.

N 个 T 类型元素占 `N*sizeof(T)` 字节。`cudaMalloc` 和 `cudaMemcpy` 的大小参数使用字节，而 `T*` 的指针加法使用元素。应分别计算传输大小和指针偏移，不能混用单位。

Repères / 复习讲义：2.4。

Q030

Prioritaire / 核心

Quelles erreurs peut provoquer un déplacement de pointeur exprimé dans la mauvaise unité ?

用错指针偏移单位会导致哪些错误?

Réponse proposée / 参考答案

Un déplacement trop grand peut viser le mauvais segment ou dépasser l'allocation. Même s'il reste dans les limites, il peut créer un trou ou un recouvrement entre les zones traitées. Il faut vérifier les intervalles de toutes les copies et de tous les kernels, pas seulement la taille totale.

偏移过大可能访问错误的数据段，也可能越过分配边界。即使仍在分配范围内，也可能使分段之间出现空隙或重叠。应检查每次复制与 kernel 使用的区间，不能只检查总分配大小。

Repères / 复习讲义：2.4。

Q031

Prioritaire / 核心

Pourquoi une allocation réussie ne garantit-elle pas une entrée valide pour le kernel ?

为什么分配成功不代表 kernel 已经获得有效输入?

Réponse proposée / 参考答案

L'allocation garantit un espace disponible, pas un contenu initialisé. Il faut remplir le buffer par une copie, une initialisation adaptée ou un calcul précédent. Si cette préparation est asynchrone, son ordre

par rapport au kernel doit aussi être garanti.

分配成功说明空间已获得，并不说明内容已经初始化。需要先通过复制、适当的初始化操作或前一个计算产生数据。如果准备过程是异步的，还必须保证它先于消费者 kernel 完成。

Repères / 复习讲义：2.4、2.11。

Q032

Prioritaire / 核心

Pourquoi conserver des données sur le GPU entre plusieurs calculs peut-il être avantageux ?

为什么让数据在多次计算之间留在 GPU 上可能更有利?

Réponse proposée / 参考答案

Cela évite des transferts CPU–GPU qui n’apportent rien au calcul. Un buffer global peut être réutilisé tant qu’il n’est pas libéré et que ses dépendances sont respectées. Il faut seulement rapatrier les données dont le CPU a réellement besoin.

这样可以避免对计算没有必要的 CPU–GPU 传输。全局缓冲区只要未释放、依赖关系正确，就能供后续 kernel 继续使用。需要回传的主要是 CPU 实际要消费的数据。

Repères / 复习讲义：2.4。

Q033

Prioritaire / 核心

Distinguez registres, mémoire locale et mémoire partagée du point de vue de la visibilité.

从可见范围区分寄存器、局部内存和共享内存。

Réponse proposée / 参考答案

Les registres et la mémoire locale contiennent des données privées à un thread. La mémoire partagée permet aux threads d’un même bloc de communiquer et de réutiliser des données. Une variable privée n’est pas forcément placée dans un registre.

寄存器和 local memory 中的数据都是线程私有的。共享内存允许同一线程块的线程交换、复用数据。线程私有变量不一定会放在寄存器里。

Repères / 复习讲义：2.3。

Q034

Prioritaire / 核心

Pourquoi le mot « locale » ne permet-il pas de conclure qu'une mémoire est rapide ?

为什么看到 local memory 不能直接认为它很快?

Réponse proposée / 参考答案

« Locale » décrit ici la portée privée au thread. Cette mémoire peut notamment contenir des tableaux privés ou des valeurs débordant des registres, avec un coût d'accès différent de celui des registres. Il faut distinguer portée logique et placement physique.

这里的 local 表示线程私有的逻辑范围。它可能存放私有数组或寄存器溢出数据，访问代价与寄存器不同。逻辑可见性和物理存放位置应分开考虑。

Repères / 复习讲义：2.3。

Q035

Prioritaire / 核心

Deux blocs placés sur le même SM partagent-ils automatiquement leurs tableaux `__shared__` ? Expliquez.

两个线程块位于同一个 SM 时，会自动共享彼此的 `__shared__` 数组吗？为什么?

Réponse proposée / 参考答案

Non, dans le modèle ordinaire du cours chaque bloc possède son instance de mémoire partagée. Le SM est une ressource matérielle, alors que la portée de l'allocation reste le bloc. Une communication entre blocs demande un mécanisme et un ordre adaptés.

在课程的普通模型中，每个块有各自的共享内存实例。SM 是硬件资源，但共享内存分配的作用域仍是线程块。块间通信需要另外选择合适的存储和顺序机制。

Repères / 复习讲义：2.3。

Q036

Prioritaire / 核心

Comparez la durée de vie d'un tableau partagé et celle d'un buffer global alloué explicitement.

比较共享内存数组与显式分配的全局缓冲区的生命周期。

Réponse proposée / 参考答案

L'instance partagée disparaît à la fin de son bloc. Une allocation globale peut survivre à plusieurs lancements jusqu'à sa libération explicite. Son contenu n'est toutefois utilisable que lorsque les opérations qui le produisent sont terminées.

共享内存实例在线程块结束后失效。显式分配的全局缓冲区可以跨多次 kernel 调用存活，直到被释放。消费其中数据之前，仍要保证生产这些数据的操作已经完成。

Repères / 复习讲义：2.3、2.4。

Q037

Complément / 补充

Dans quel cas la mémoire constante est-elle intéressante, et quelle limite faut-il retenir ?

常量内存在哪种情况下有利？需要记住什么限制？

Réponse proposée / 参考答案

Elle est adaptée à des données mises à jour depuis l’hôte et lues par les kernels. Les threads d’un warp lisant la même adresse peuvent bénéficier de la diffusion. Des lectures à des adresses différentes peuvent réduire cet avantage ; ce n’est pas une accélération automatique de tout tableau en lecture seule.

它适合由主机更新、供 kernel 读取的数据。同一 warp 的线程读取相同地址时可以受益于广播。读取不同地址可能削弱收益，因此不能认为把任意只读数组放入常量内存都会加速。

Repères / 复习讲义：2.3。

Q038

Prioritaire / 核心

Définissez la coalescence sans supposer un nombre fixe de transactions.

在不假设固定事务数的前提下，解释合并访存。

Réponse proposée / 参考答案

Le matériel regroupe les accès globaux des threads actifs d’un warp en transactions mémoire. Des adresses proches et correctement alignées permettent généralement de solliciter moins de segments. Le nombre exact dépend aussi des types, de l’alignement et de l’architecture.

硬件把一个 warp 中活跃线程的全局内存访问组织为内存事务。地址邻近且对齐适当时，通常能减少涉及的内存段。确切事务数还受数据类型、对齐和架构影响。

Repères / 复习讲义：2.7。

Q039

Prioritaire / 核心

Pourquoi deux accès contigus d'un seul thread ne suffisent-ils pas à démontrer une bonne coalescence ?

为什么一个线程连续访问两个相邻元素，不足以证明合并访存良好?

Réponse proposée / 参考答案

La coalescence concerne une instruction mémoire exécutée par plusieurs threads d'un warp. Il faut comparer les adresses demandées par ces threads au même stade. Les accès successifs d'un thread peuvent être contigus alors que les adresses entre threads sont très dispersées.

合并访存关注一个 warp 中多个线程执行同一条访存指令时的地址关系。需要比较这些线程在同一阶段请求的地址。单个线程前后访问连续，线程之间的地址仍可能很分散。

Repères / 复习讲义：2.7。

Q040

Prioritaire / 核心

Pour une matrice stockée par lignes, pourquoi faire varier la colonne entre lanes voisines est-il souvent favorable ?

对于按行存储的矩阵，为什么让相邻 lane 对应相邻列通常更有利?

Réponse proposée / 参考答案

L'indice d'un élément vaut `row*N+col` . À ligne fixe, une colonne supplémentaire décale l'adresse d'un seul élément ; à colonne fixe, une ligne supplémentaire la décale de N éléments. Le premier motif favorise donc souvent la coalescence.

元素下标为 `row*N+col` 。行号固定时，列号增加一只跨过一个元素；列号固定时，行号增加一要跨过 N 个元素。因此前一种映射通常更利于合并访存。

Repères / 复习讲义：2.7。

Q041

Prioritaire / 核心

Comment la mémoire partagée peut-elle réduire le trafic en mémoire globale ?

共享内存如何帮助减少全局内存流量?

Réponse proposée / 参考答案

Les threads d'un bloc peuvent charger une donnée une fois puis la réutiliser à plusieurs reprises. Cela peut éviter des chargements globaux redondants, avec les synchronisations nécessaires. Le bénéfice dépend de la réutilisation obtenue et du coût supplémentaire introduit.

线程块可以先加载一份数据，再在块内多次复用，从而减少重复的全局内存读取。使用时要建立必要的同步。是否获益取决于减少的流量能否抵消额外搬运和同步的成本。

Repères / 复习讲义：2.7、2.8。

Q042

Complément / 补充

Distinguez un conflit de banques en mémoire partagée d’un accès global non coalescé.

区分共享内存 bank conflict 与全局内存访问不合并。

Réponse proposée / 参考答案

Un accès global non coalescé peut demander davantage de segments ou transactions. Un conflit de banques concerne des adresses distinctes servies par une même banque partagée lors d’une opération. Les deux dépendent du motif d’adresses, mais affectent des mécanismes mémoire différents.

全局访问不合并可能增加涉及的内存段或事务数。bank conflict 则是同一次操作中，不同共享内存地址需要由同一 bank 服务。两者都与地址模式有关，但作用于不同的存储机制。

Repères / 复习讲义：2.7、2.8。

Q043

Complément / 补充

Pourquoi le padding d’un tableau partagé peut-il réduire les conflits de banques ?

为什么为共享内存数组增加填充，可能减少 bank conflict?

Réponse proposée / 参考答案

Le padding modifie la distance entre les lignes et donc les banques utilisées. Dans le modèle de 32 banques et de mots de 4 octets, passer de `float[32][32]` à `float[32][33]` répartit mieux une lecture de colonne. Il faut vérifier le motif réel plutôt que supposer que tout padding améliore les performances.

填充会改变行间距，进而改变地址映射到的 bank。在 32 个 bank、每字 4 字节的简化模型中，把 `float[32][32]` 改为 `float[32][33]` 能分散按列读取的 bank。仍应检查实际访问模式，不能假设任意填充都有收益。

Repères / 复习讲义：2.8。

Q044

Complément / 补充

Pourquoi diminuer le nombre de registres par thread peut-il parfois ralentir un kernel ?

为什么减少每线程寄存器数有时反而会使 kernel 变慢?

Réponse proposée / 参考答案

Une limite plus basse peut permettre davantage de résidence, mais elle peut aussi provoquer des débordements en mémoire locale. Le surcoût de ces accès peut dépasser le gain de parallélisme. Il faut donc comparer les performances et pas seulement le nombre de registres.

更低的寄存器限制可能增加驻留线程数量，也可能导致数据溢出到 local memory。新增访问的代价可能超过并行度提高的收益，因此不能仅凭寄存器数判断性能。

Repères / 复习讲义：2.9。

Q045

Complément / 补充

Pourquoi une occupation plus élevée ne garantit-elle pas un kernel plus rapide ?

为什么更高的驻留率不保证 kernel 更快?

Réponse proposée / 参考答案

L’occupation mesure les warps résidents relativement au maximum du SM. Elle ne mesure ni la quantité de travail utile ni directement le débit mémoire ou le temps d’exécution. Davantage de résidence peut aider à masquer la latence, mais aussi modifier les compromis de registres et de mémoire partagée.

驻留率表示实际驻留 warp 数相对于 SM 上限的比例，并不直接衡量有效工作量、内存吞吐或运行时间。增加驻留可能有助于隐藏延迟，但也可能改变寄存器和共享内存的资源权衡。

Repères / 复习讲义：2.9。

Q046

Complément / 补充

Qu’est-ce que le pitch, et pourquoi peut-il dépasser la largeur utile d’une ligne ?

什么是 pitch，为什么它可以大于一行的有效宽度?

Réponse proposée / 参考答案

Le pitch est la distance en octets entre les débuts de deux lignes consécutives. L'allocation peut ajouter un padding pour satisfaire des contraintes d'alignement. La largeur utile reste la quantité de données à copier dans chaque ligne.

pitch 是相邻两行起始地址之间的字节距离。分配器可能为满足对齐约束添加行末填充。每行实际需要复制的数据量仍由有效宽度决定。

Repères / 复习讲义：2.5。

Q047

Complément / 补充

Comment raisonner sur l'adresse d'un élément d'une matrice avec pitch ?

如何计算带 pitch 的矩阵中某个元素的地址?

Réponse proposée / 参考答案

On décale d'abord la base de `row*pitch` octets pour atteindre la ligne. On convertit ensuite cette adresse vers le type des éléments et on applique l'indice de colonne. Ajouter directement `row*pitch` à un pointeur typé multiplierait encore le déplacement par la taille du type.

先把基址按字节移动 `row*pitch`，得到该行起点，再转换成元素类型指针并按列下标访问。若直接对元素类型指针加 `row*pitch`，会额外乘一次元素大小。

Repères / 复习讲义：2.5。

Q048

Complément / 补充

La valeur `float4` garantit-elle une accélération ? Que faut-il examiner ?

使用 `float4` 是否保证加速？还要检查什么?

Réponse proposée / 参考答案

`float4` est un type avec des contraintes d'alignement ; son utilisation ne garantit pas une instruction arithmétique vectorielle ni un gain. Il faut examiner les adresses, la taille utile des transferts et le code généré. La coalescence entre threads reste aussi à considérer.

`float4` 是带有对齐要求的类型，使用它不保证生成向量算术指令，也不保证更快。还需检查地址、有效传输量和生成代码，并考虑线程之间的访存是否合并。

Repères / 复习讲义：2.2。

Q049

Complément / 补充

Pourquoi faut-il vérifier la capacité réellement allouée même si la taille d'une copie paraît correcte ?

即使一次复制的长度看起来正确，为什么仍要检查实际分配容量?

Réponse proposée / 参考答案

La validité dépend à la fois du début du segment et de sa longueur. Une copie de N éléments peut dépasser le buffer si elle commence trop loin. Il faut comparer l'intervalle complet à la capacité de la source et à celle de la destination.

合法性由区间起点和长度共同决定。即使复制 N 个元素的长度正确，起点过大仍会越界。需要把完整访问区间同时与源和目标的容量比较。

Repères / 复习讲义：2.4。

Q050

Prioritaire / 核心

Pourquoi un type de mémoire ou un cache ne suffit-il pas à assurer la communication correcte entre threads ?

为什么仅选择某种内存或缓存，不能保证线程间通信正确?

Réponse proposée / 参考答案

Il faut d'abord que le stockage soit accessible aux threads concernés : dans le modèle ordinaire, `__shared__` se limite au bloc, et la mémoire locale reste privée. Une mémoire accessible ne garantit pas que la production précède la lecture. Il faut un ordre et une synchronisation adaptés à la portée de la communication, sans accès concurrents incompatibles.

首先，存储空间必须能被参与通信的线程访问：在普通线程块模型中，`__shared__` 的范围是一个块，local memory 仍然是线程私有的。能够访问某块内存，并不保证生产者先于消费者完成写入。还需建立与通信范围相符的执行顺序和同步，避免不兼容的并发读写。

Repères / 复习讲义：2.3、2.10。

Asynchronisme et synchronisation / 异步执行与同步

Q051

Prioritaire / 核心

Expliquez ce que signifie « lancement asynchrone par rapport au CPU » et ce que cette expression ne permet pas de conclure sur les opérations du GPU.

解释“相对于 CPU 的异步启动”是什么意思，以及它不能说明 GPU 操作之间有什么关系。

Réponse proposée / 参考答案

Le CPU peut poursuivre son programme sans attendre la fin du noyau lancé. Cela ne signifie pas que toutes les opérations GPU s’exécutent simultanément : celles d’un même stream restent ordonnées. Il faut donc distinguer le retour de l’appel CPU et l’achèvement du calcul GPU.

CPU 可以在不等待刚启动的 kernel 完成的情况下继续执行。它不表示所有 GPU 操作都会同时执行，同一 stream 内的操作仍有先后顺序。因此，应区分 CPU 调用返回和 GPU 计算完成。

Repères / 复习讲义：2.11。

Q052

Prioritaire / 核心

Dans une chaîne « transfert d’entrée → calcul GPU → transfert de sortie → utilisation CPU », quelles sont les deux catégories de dépendances à vérifier ?

在“输入传输 → GPU 计算 → 输出传输 → CPU 使用结果”这一流程中，需要检查哪两类依赖?

Réponse proposée / 参考答案

Il faut d’abord garantir l’ordre des opérations GPU : le calcul doit recevoir ses entrées, puis la copie de sortie doit attendre les résultats. Un même stream peut établir cet ordre. Il faut ensuite que le CPU attende la fin de la copie de sortie avant de lire le buffer hôte.

首先要保证 GPU 操作之间的顺序：计算先获得输入，输出复制再等待计算结果；同一 stream 可以建立这种顺序。其次要保证 CPU 在输出复制完成后才读取主机缓冲区。这两类依赖需要分别检查。

Repères / 复习讲义：2.11。

Q053

Prioritaire / 核心

Deux noyaux sont placés dans le même stream, le second utilisant le résultat du premier. Pourquoi une synchronisation du CPU entre les deux lancements est-elle généralement inutile pour cette dépendance ?

两个 kernel 放在同一 stream 中，后者使用前者的结果。为什么通常无需为了这条依赖，在两次启动之间让 CPU 同步等待?

Réponse proposée / 参考答案

L’ordre du stream garantit que le second noyau utilise les résultats après l’exécution du premier. Le CPU peut donc soumettre les deux travaux sans attendre entre les appels. Une attente CPU reste nécessaire lorsqu’une opération CPU doit elle-même utiliser un résultat qui n’est pas encore disponible.

同一 stream 的顺序保证后一个 kernel 在前一个执行结束后使用其结果。因此，CPU 可以连续提交两项工作，不必在两次调用之间等待。如果接下来是 CPU 自己使用尚未就绪的结果，仍然需要相应等待。

Repères / 复习讲义：2.11。

Q054

Complément / 补充

Le CPU soumet successivement deux travaux indépendants dans deux streams distincts. Pourquoi l’ordre des appels ne suffit-il pas à prédire leur ordre d’achèvement ?

CPU 先后向两个不同的 stream 提交独立工作。为什么调用的先后顺序不足以预测它们的完成顺序?

Réponse proposée / 参考答案

En l’absence de dépendance entre les streams, les soumissions successives n’imposent pas un ordre complet d’exécution entre les travaux GPU. Leur durée et l’ordonnancement peuvent conduire le second travail soumis à finir en premier. Le matériel peut aussi les faire se chevaucher si les ressources le permettent.

如果两个 stream 之间没有依赖，CPU 先后提交并不建立 GPU 工作之间的完整执行顺序。任务耗时和调度可能使后提交的工作先完成。硬件和资源允许时，它们也可能重叠执行。

Repères / 复习讲义：2.11。

Q055

Prioritaire / 核心

Quel est l’intérêt d’utiliser plusieurs streams dans un traitement par lots de données, et quelles conditions limitent le recouvrement ?

分批处理数据时，使用多个 stream 有什么作用？哪些条件会限制重叠执行?

Réponse proposée / 参考答案

Plusieurs streams permettent de soumettre des chaînes de travail indépendantes, par exemple la copie d’un lot pendant le calcul d’un autre. Il faut respecter les dépendances et disposer de buffers qui ne se gênent pas. Le recouvrement dépend aussi des capacités du GPU, des ressources disponibles et, pour les transferts hôte asynchrones, de la mémoire hôte utilisée.

多个 stream 可以提交相互独立的工作流程，例如计算一批数据时传输另一批。必须正确处理依赖，并避免不同批次的缓冲区相互冲突。实际重叠还受 GPU 能力、可用资源以及异步主机传输所用内存类型的限制。

Repères / 复习讲义：2.11。

Q056

Complément / 补充

Pourquoi séparer les données en plusieurs lots ne suffit-il pas à rendre automatiquement sûr un programme utilisant plusieurs streams ?

为什么把数据拆成多个批次，还不足以自动保证多 stream 程序安全?

Réponse proposée / 参考答案

Il faut vérifier les plages mémoire réellement lues et écrites par chaque travail, pas seulement les noms des lots. Des offsets erronés ou un buffer réutilisé trop tôt peuvent créer un chevauchement entre opérations concurrentes. Les dépendances de production et de consommation doivent également être établies entre streams lorsque des données sont partagées.

需要检查每项工作实际读写的内存范围，不能只看批次名称。错误的偏移或过早复用缓冲区，都可能使并发操作访问重叠区域。如果不同 stream 共享生产者和消费者数据，还必须建立相应依赖。

Repères / 复习讲义：2.11。

Q057

Prioritaire / 核心

Comparez `cudaStreamSynchronize(s)` et `cudaDeviceSynchronize()` pour un programme qui utilise plusieurs streams.

对于使用多个 stream 的程序，比较 cudaStreamSynchronize(s) 和 cudaDeviceSynchronize()。

Réponse proposée / 参考答案

`cudaStreamSynchronize(s)` fait attendre le CPU jusqu'à l'achèvement des travaux soumis dans le stream indiqué. `cudaDeviceSynchronize()` attend les travaux précédemment soumis dans le contexte courant sur le device courant. Une attente limitée au stream utile évite de demander explicitement l'achèvement de travaux indépendants des autres streams.

cudaStreamSynchronize(s) 使 CPU 等待指定 stream 中已提交的工作完成。cudaDeviceSynchronize() 等待当前 device 当前 context 中此前提交的工作。只等待所需 stream，可以避免显式要求其他 stream 中的独立工作也全部完成。

Repères / 复习讲义：2.11。

Q058

Prioritaire / 核心

Pour faire exécuter un consommateur après un producteur situé dans un autre stream, comparez une attente CPU avant le lancement du consommateur et une dépendance par événement.

为了让一个 stream 中的消费者在另一个 stream 的生产者之后执行，比较“CPU 等待后再启动消费者”和“使用事件建立依赖”这两种方法。

Réponse proposée / 参考答案

Le CPU peut attendre la fin du producteur, puis soumettre le consommateur : l’ordre est alors correct, mais le CPU attend entre les deux. Avec un événement enregistré après le producteur, le stream consommateur peut attendre cet événement avant son calcul. Cette seconde méthode exprime la dépendance dans les files GPU et permet au CPU de poursuivre ses soumissions.

CPU 可以先等待生产者完成，再提交消费者，这能够保证顺序，但 CPU 在中间需要等待。也可以在生产者之后记录事件，让消费者 stream 在计算前等待该事件。后一种方法把依赖写入 GPU 队列，使 CPU 可以继续提交其他工作。

Repères / 复习讲义：2.11。

Q059

Prioritaire / 核心

Distinguez le rôle de `cudaEventRecord`, de `cudaEventSynchronize` et de `cudaStreamWaitEvent`.

区分 cudaEventRecord、cudaEventSynchronize 和 cudaStreamWaitEvent 的作用。

Réponse proposée / 参考答案

`cudaEventRecord` place un repère d’avancement dans un stream. `cudaEventSynchronize` fait attendre le thread CPU jusqu’à l’achèvement de l’événement. `cudaStreamWaitEvent` impose aux opérations suivantes d’un stream d’attendre cet événement, sans demander au CPU d’attendre sa fin.

cudaEventRecord 在 stream 中放置一个执行进度标记。cudaEventSynchronize 让 CPU 线程等待事件完成。cudaStreamWaitEvent 让指定 stream 的后续操作等待该事件，而不要求 CPU 等待事件结束。

Repères / 复习讲义：2.11、2.12。

Q060

Prioritaire / 核心

Pourquoi faut-il enregistrer l'événement après le producteur et placer l'attente avant le consommateur ?

为什么应当在生产者之后记录事件，并在消费者之前放置事件等待?

Réponse proposée / 参考答案

L'événement enregistré après le producteur ne s'achève qu'après les opérations qui le précèdent dans ce stream. L'attente placée avant le consommateur empêche celui-ci d'utiliser trop tôt les résultats. Si l'événement est enregistré avant le producteur, son achèvement ne prouve pas que les données ont été produites.

生产者之后记录的事件，要等该 stream 中此前的操作完成后才完成。消费者之前的等待可以防止它过早使用结果。如果事件记录在生产者之前，事件完成就不能证明所需数据已经生成。

Repères / 复习讲义：2.11。

Q061

Prioritaire / 核心

Expliquez le principe de mesure d'un noyau avec deux événements CUDA, sans écrire de programme.

不写程序，说明如何使用两个 CUDA event 测量一个 kernel 的耗时。

Réponse proposée / 参考答案

On enregistre un événement de début avant le noyau et un événement de fin après celui-ci, dans le même stream. On attend l'achèvement de l'événement de fin, puis `cudaEventElapsedTime` fournit la durée entre les deux événements en millisecondes. La mesure porte sur l'intervalle encadré et n'inclut pas automatiquement les transferts placés ailleurs.

在同一个 stream 中，kernel 之前记录开始事件，之后记录结束事件。等待结束事件完成，再用 cudaEventElapsedTime 取得两事件之间的时间，单位为毫秒。测量范围由两个事件的位置决定，不会自动包含放在区间外的传输。

Repères / 复习讲义：2.12。

Q062

Complément / 补充

Pourquoi les durées « noyau seul » et « traitement complet avec transferts » peuvent-elles être très différentes ? Comment décrire correctement une mesure ?

为什么“仅 kernel 时间”和“包含传输的完整处理时间”可能相差很大？报告时间时应如何说明测量范围?

Réponse proposée / 参考答案

Le traitement complet peut inclure les copies H→D et D→H, les attentes CPU et éventuellement l'initialisation, en plus du calcul. Il faut préciser les opérations comprises entre le début et la fin de la mesure, ainsi que l'inclusion ou non d'une première exécution. Avec un chronomètre CPU, le travail mesuré doit être terminé avant de prendre l'horodatage final.

完整处理除计算外，可能还包括 H→D、D→H 复制、CPU 等待及初始化。报告时应明确起止点包含哪些操作，以及是否包含首次执行。使用 CPU 计时器时，必须在被测工作完成后再记录结束时间。

Repères / 复习讲义：2.12。

Q063

Prioritaire / 核心

Pourquoi l'utilisation d'un buffer hôte pinned ne dispense-t-elle pas de synchroniser avant de lire un résultat reçu du GPU ?

为什么使用 pinned 主机缓冲区之后，读取 GPU 回传结果前仍然可能需要同步？

Réponse proposée / 参考答案

Le verrouillage des pages rend le buffer adapté aux transferts hôte-GPU asynchrones. Il ne signifie pas que le transfert est déjà achevé lorsque l'appel retourne. Le CPU doit établir la fin de la copie D→H avant de lire ou de réutiliser sa destination.

页锁定使缓冲区适用于主机与 GPU 之间的异步传输。它不表示调用返回时传输已经完成。CPU 在读取或复用 D→H 目标缓冲区前，仍需确认复制结束。

Repères / 复习讲义：2.11。

Q064

Prioritaire / 核心

Comparez les précautions à prendre sur un buffer hôte pendant une copie H→D et pendant une copie D→H.

比较 H→D 复制与 D→H 复制进行期间，对主机缓冲区分别需要注意什么。

Réponse proposée / 参考答案

Pendant une copie H→D, le buffer hôte est une source : il doit rester valide et ne pas être modifié pendant sa lecture par le transfert. Pendant une copie D→H, il est une destination : le CPU ne doit pas l'utiliser comme un résultat déjà disponible avant la fin de l'écriture. Dans les deux cas, une libération ou une réutilisation prématurée compromet la sûreté.

H→D 时，主机缓冲区是源，它在被传输读取期间应保持有效且不被修改。D→H 时，主机缓冲区是目标，CPU 不能在写入完成前把它当作已经就绪的结果。两种情况下，过早释放或复用都会破坏安全性。

Repères / 复习讲义：2.11。

Q065

Complément / 补充

Un programme multi-stream donne le bon résultat lors de plusieurs essais. Pourquoi cela ne suffit-il pas à prouver que ses synchronisations sont correctes ?

一个多 stream 程序连续几次都得到正确结果。为什么这不足以证明它的同步正确?

Réponse proposée / 参考答案

Une dépendance manquante peut rester invisible lorsque l’ordonnancement fait finir le producteur assez tôt par hasard. Une autre taille de données ou une autre charge du GPU peut révéler la lecture prématurée. Il faut vérifier que chaque dépendance est garantie par l’ordre d’un stream ou par une synchronisation explicite correctement placée.

缺失的依赖可能因为某次调度恰好让生产者较早完成而未表现为错误。换一种数据规模或 GPU 负载，就可能暴露过早读取。需要逐项确认依赖是否由同流顺序或正确放置的显式同步来保证。

Repères / 复习讲义：2.11。

Q066

Complément / 补充

Expliquez pourquoi ajouter une synchronisation globale à la fin d’un programme ne corrige pas nécessairement une course entre deux noyaux de streams différents.

解释为什么在程序末尾补一次全局同步，不一定能够修复不同 stream 中两个 kernel 之间的数据竞争。

Réponse proposée / 参考答案

La synchronisation finale attend que les travaux déjà soumis soient terminés. Elle ne modifie pas leur ordre passé et ne recommence pas un consommateur qui a lu trop tôt. La dépendance doit empêcher la lecture prématurée avant qu’elle ne se produise.

末尾的同步等待此前提交的工作全部结束。它不会改变已经发生的执行顺序，也不会重新运行过早读取的消费者。所需依赖必须在读取发生前就阻止这种过早访问。

Repères / 复习讲义：2.11。

Q067

Complément / 补充

Si A précède B et B précède C par des dépendances correctement établies, que peut-on déduire lorsque le CPU a confirmé la fin de C ? Quelle limite faut-il garder à l'esprit ?

如果已正确建立 A 先于 B、B 先于 C 的依赖，CPU 确认 C 完成后可以推断什么？还需要注意什么限制?

Réponse proposée / 参考答案

La fin de C implique la fin des opérations A et B dont C dépend. Attendre l'extrémité de cette chaîne peut donc suffire pour les données produites le long de celle-ci. Cela ne prouve pas la fin de travaux indépendants qui ne sont pas reliés à cette chaîne.

C 完成意味着它所依赖的 A、B 也已完成。因此，对这条依赖链中产生的数据，等待链末端可能已经足够。但这不能证明与该依赖链无关的独立工作也已结束。

Repères / 复习讲义：2.11。

Q068

Prioritaire / 核心

Pourquoi la visibilité d'un tableau shared par tous les threads d'un bloc ne suffit-elle pas à garantir la sûreté d'un échange de données ?

既然 shared 数组可以被同一个 block 的所有线程访问，为什么线程间交换数据仍然需要考虑同步?

Réponse proposée / 参考答案

La possibilité d'accéder à la même mémoire ne garantit pas que le thread producteur a déjà effectué son écriture. Une lecture peut donc arriver avant la production de la valeur attendue. Une barrière de bloc correctement placée établit le rendez-vous et la visibilité des écritures précédentes concernées avant les lectures suivantes.

能够访问同一片内存，不代表生产者线程已经完成写入。读取可能早于所需数据的产生。正确放置的 block 屏障既让线程会合，也保证相关先前写入对屏障后的读取可见。

Repères / 复习讲义：2.10。

Q069

Prioritaire / 核心

Lorsqu'un bloc traite le dernier morceau d'un tableau dont la taille n'est pas un multiple de la taille du bloc, comment préserver la participation à une barrière collective ?

当数组长度不是 block 大小的整数倍、最后一个 block 含越界线程时，如何保证集体屏障的参与正确?

Réponse proposée / 参考答案

On sépare la validité de l’accès mémoire de la participation à la barrière. Les threads hors limites évitent l’accès invalide et peuvent fournir une valeur neutre, par exemple zéro pour une somme. Tous les threads du bloc suivent ensuite le même passage par la barrière, au lieu de la placer seulement dans la branche des indices valides.

应当把“能否访问对应数组元素”和“是否参与屏障”分开处理。越界线程避免非法访问，并可提供中性值，例如求和时使用零。随后让整个 block 的线程一致地通过屏障，不把屏障仅放在有效下标的分支中。

Repères / 复习讲义：2.10。

Q070

Complément / 补充

Pourquoi `__syncthreads()` ne permet-il pas, dans un noyau ordinaire, d’attendre les résultats produits par tous les autres blocs ? Quelle organisation simple établit deux phases globales ?

为什么普通 kernel 中的 __syncthreads() 不能等待所有其他 block 产生结果？一种简单的全局两阶段组织方式是什么?

Réponse proposée / 参考答案

`__syncthreads()` ne synchronise que les threads du bloc courant. Pour deux phases globales, on peut utiliser deux noyaux successifs dans le même stream : le second commence après la fin du premier. Cette organisation établit une frontière globale entre les phases sans prétendre synchroniser toute la grille par une barrière de bloc.

__syncthreads() 只同步当前 block 内的线程。对于全局两阶段计算，可以在同一 stream 中依次启动两个 kernel，使第二个在第一个全部完成后开始。这种组织方式通过两次启动建立全局阶段边界。

Repères / 复习讲义：2.10、2.11。

Q071

Prioritaire / 核心

Expliquez, avec le principe lecture–modification–écriture, comment deux threads peuvent perdre un incrément sur un compteur partagé.

用“读取—修改—写回”的过程说明：两个线程为何可能在共享计数器上丢失一次递增?

Réponse proposée / 参考答案

Les deux threads peuvent lire la même ancienne valeur, par exemple 5. Chacun calcule 6 puis écrit 6, si bien que le compteur n'augmente que d'une unité au lieu de deux. Une opération atomique adaptée protège chaque mise à jour complète contre cette interférence.

两个线程可能都先读到同一个旧值，例如 5。各自算出 6 后写回，最终计数器只增加了一次，而不是两次。适当的原子操作可以保护完整的单次更新，避免这种相互覆盖。

Repères / 复习讲义：2.10。

Q072

Prioritaire / 核心

Pourquoi un `atomicAdd` ne remplace-t-il pas `__syncthreads()`, et pourquoi la barrière ne suffit-elle pas à rendre sûrs des incréments concurrents ordinaires ?

为什么 atomicAdd 不能替代 __syncthreads()？为什么仅有屏障也不能让普通并发递增变得安全?

Réponse proposée / 参考答案

`atomicAdd` protège une mise à jour de son adresse cible, mais ne fait pas attendre tous les threads à un rendez-vous collectif. `__syncthreads()` sépare des phases du bloc, mais des incréments ordinaires situés dans une même phase peuvent toujours se chevaucher. Ces outils répondent donc à des besoins différents : indivisibilité d'une mise à jour et coordination entre phases.

atomicAdd 保护目标地址上的一次更新，但不会让所有线程在某处集体会合。__syncthreads() 可以分隔 block 内的执行阶段，可同一阶段中的普通递增仍可能并发冲突。两者分别解决更新的不可分割性和阶段间协调问题。

Repères / 复习讲义：2.10。

Q073

Complément / 补充

Pourquoi un grand nombre d'opérations atomiques sur une seule adresse peut-il limiter les performances ? Quelle idée générale permet de réduire cette pression ?

为什么大量原子操作集中到同一个地址可能限制性能？什么总体思路可以减轻这种压力?

Réponse proposée / 参考答案

Les mises à jour concurrentes d'une même adresse sont en contention, même si elles sont correctes grâce à l'atomicité. On peut combiner localement plusieurs contributions, puis effectuer moins de mises à jour atomiques sur le résultat commun. Pour des valeurs flottantes, ce regroupement change éventuellement l'ordre des additions et donc les arrondis.

即使原子性保证更新正确，大量线程同时更新一个地址仍会产生争用。可以先在局部合并若干贡献，再减少对公共结果的原子更新次数。如果使用浮点数，这种合并还可能改变加法顺序和舍入结果。

Repères / 复习讲义：2.10。

Q074

Complément / 补充

Que faut-il répondre à l’affirmation : « Le tableau est volatile, donc les threads peuvent l’utiliser sans autre synchronisation » ?

如何评价“数组已经声明为 volatile，所以线程可以不做其他同步就直接使用它”这一说法?

Réponse proposée / 参考答案

Cette affirmation est fausse : `volatile` impose des contraintes au traitement des accès par le compilateur. Il ne garantit ni une mise à jour atomique ni l’arrivée des autres threads à un point donné. Il faut établir séparément les dépendances de lecture et d’écriture avec les mécanismes adaptés.

这个说法不正确：volatile 对编译器处理访存的方式施加约束。它既不保证更新具有原子性，也不保证其他线程已经到达某个位置。读写之间的依赖仍需用适当机制单独建立。

Repères / 复习讲义：2.10。

Q075

Complément / 补充

Proposez une méthode courte pour analyser la sûreté d’un programme qui combine plusieurs streams, des copies asynchrones et une lecture finale sur le CPU.

给出一种简短方法，用来检查包含多个 stream、异步复制和最后 CPU 读取的程序是否安全。

Réponse proposée / 参考答案

Pour chaque buffer, on identifie son emplacement, sa plage valide, ses producteurs et ses consommateurs. On vérifie ensuite que chaque lecture attend la production correspondante, par l’ordre du stream ou par une dépendance explicite. Enfin, on contrôle que le CPU attend le transfert final et qu’aucun buffer n’est modifié, libéré ou réutilisé avant sa dernière utilisation.

先对每个缓冲区确认存放位置、有效范围、生产者和消费者。再检查每次读取是否通过同流顺序或显式依赖，等待相应生产过程完成。最后确认 CPU 等待了最终回传，并且没有缓冲区在最后一次使用结束前被修改、释放或复用。

Repères / 复习讲义：2.11。

TD, TP et bases des API / TD、TP 与 API 基础

Q076

Prioritaire / 核心

Pourquoi un fichier CUDA peut-il nécessiter à la fois une compilation pour le CPU et une compilation pour le GPU ?

为什么一个 CUDA 源文件可能同时需要面向 CPU 和 GPU 的两条编译路径?

Réponse proposée / 参考答案

Un fichier CUDA peut contenir du code hôte, comme `main`, et du code destiné au GPU, comme les kernels. `nvcc` organise leur traitement pour leurs cibles respectives, en s'appuyant notamment sur un compilateur hôte. Dans l'exemple du TP1, le lancement d'un kernel est déclenché par le code hôte, tandis que son corps est exécuté par les threads GPU.

一个 CUDA 文件可以同时包含 `main` 等主机代码和 kernel 等 GPU 代码。`nvcc` 为它们组织不同目标的编译过程，其中主机部分会用到主机编译器。在 TP1 示例中，启动 kernel 的动作由主机代码发起，kernel 函数体则由 GPU 线程执行。

Repères / 复习讲义：4.6。

Q077

Complément / 补充

Quelle différence faites-vous entre un fichier PTX, un cubin et un fatbin ?

PTX、cubin 与 fatbin 文件分别是什么，它们有什么区别?

Réponse proposée / 参考答案

Le PTX est une représentation intermédiaire textuelle du code GPU, tandis qu'un cubin contient du code machine pour une cible GPU donnée. Un fatbin est un conteneur pouvant regrouper du PTX et/ou des cubins. Il peut ainsi embarquer plusieurs formes du code pour différentes cibles.

PTX 是 GPU 代码的文本中间表示，cubin 是针对某个 GPU 目标的机器码。fatbin 是一个容器，可以打包 PTX 和／或 cubin。因此它能够携带面向不同目标的多种代码形式。

Repères / 复习讲义：4.6。

Q078

Prioritaire / 核心

Le sujet ancien mentionne `compute_30` et `sm_30`. Supposons qu'une autre compilation produise un fichier ciblant `sm_90` : comment interpréter ces noms sans confondre les versions ?

旧题提到 `compute_30`、`sm_30`。假设另一次编译生成了以 `sm_90` 为目标的文件，怎样理解这些名字，避免混淆版本?

Réponse proposée / 参考答案

Les nombres désignent la capacité de calcul ciblée : 30 correspond à 3.0 et 90 à 9.0, pas à la version du CUDA Toolkit. `compute` indique une cible virtuelle et `sm` une cible machine. Les fichiers réellement produits dépendent des options et du compilateur ; le suffixe d'un ancien sujet n'est pas un résultat attendu dans tout environnement.

这些数字表示目标计算能力：30 对应 3.0，90 对应 9.0，并非 CUDA Toolkit 的版本。`compute` 表示虚拟目标，`sm` 表示机器目标。实际文件取决于编译选项和编译器，不能要求所有环境都生成旧题中的后缀。

Repères / 复习讲义：4.6。

Q079

Complément / 补充

Pourquoi la réussite de `nvcc -c programme.cu` ne signifie-t-elle pas encore que l'on dispose d'un programme exécutable complet ?

为什么 `nvcc -c programme.cu` 成功，还不能说明已经获得一个完整可执行程序?

Réponse proposée / 参考答案

L'option `-c` produit un fichier objet et ne réalise pas l'édition de liens finale. Il reste à assembler les objets et les bibliothèques nécessaires pour obtenir l'exécutable. La réussite de la compilation ne prouve pas non plus que le programme donnera un résultat correct à l'exécution.

`-c` 生成目标文件，不执行最终链接。还需要将目标文件和所需库链接起来，才能得到完整可执行文件。编译成功本身也不能证明程序运行结果正确。

Repères / 复习讲义：4.6。

Q080

Prioritaire / 核心

Dans le GEMM naïf du TP1, pourquoi peut-on affecter un élément de C à chaque thread sans synchronisation entre threads ?

TP1 的朴素 GEMM 中，为什么可以让每个线程计算 C 的一个元素，而无需在线程之间同步?

Réponse proposée / 参考答案

Chaque thread lit une ligne de A et une colonne de B, puis accumule le produit scalaire dans une variable privée. Si A, B et C ne se recouvrent pas et qu'un seul thread écrit chaque élément de C, les threads ne partagent aucun résultat intermédiaire. Aucune barrière entre threads n'est alors nécessaire pour ce calcul, mais le consommateur du résultat doit toujours attendre sa disponibilité.

每个线程读取 A 的一行和 B 的一列，在自己的局部变量中累加内积。若 A、B、C 不重叠，且每个 C 元素仅由一个线程写入，线程之间就不交换中间结果。因此这项计算无需块内屏障，不过后续使用结果的操作仍需等待结果就绪。

Repères / 复习讲义：4.7。

Q081

Prioritaire / 核心

Pourquoi faut-il restaurer C avant chaque répétition chronométrée de `C ← αAB + C` si l'on veut répéter exactement le même problème ?

对 `C ← αAB + C` 重复计时，若希望每轮都计算同一个问题，为什么需要在每轮之前恢复 C?

Réponse proposée / 参考答案

Cette opération lit puis modifie C. Sans restauration, la deuxième exécution repart du résultat de la première et ajoute de nouveau `αAB`, ce qui change le problème initial. Il faut donc restaurer C avant chaque essai et préciser si cette restauration appartient ou non au périmètre mesuré.

该运算既读取 C，也会修改 C。不恢复时，第二轮从第一轮结果继续累加 `αAB`，已经不是相同初值的问题。因此应在每轮前恢复 C，并明确恢复操作是否包含在计时范围内。

Repères / 复习讲义：4.8。

Q082

Prioritaire / 核心

Expliquez les deux accès `A[row*N+k]` et `B[k*N+col]` dans un produit de matrices carrées stockées par lignes.

方阵按行存储时，怎样解释矩阵乘法中的 `A[row*N+k]` 和 `B[k*N+col]` 两个下标?

Réponse proposée / 参考答案

En stockage par lignes, l'élément `(i,j)` se trouve à l'indice `i*N+j`. Pour calculer `C[row,col]`, on multiplie les éléments de la ligne `row` de A par ceux de la colonne `col` de B. Quand k varie, le premier accès parcourt cette ligne et le second cette colonne.

行主序中，元素 `(i,j)` 的一维下标是 `i*N+j` 。计算 `C[row,col]` 时，需要将 A 的第 row 行与 B 的第 col 列对应相乘。k 变化时，第一个下标沿这行移动，第二个下标沿这列移动。

Repères / 复习讲义：4.7。

Q083

Prioritaire / 核心

Quel est le rôle de α et β dans `C ← αAB + βC` ? Que deviennent les données d'entrée nécessaires lorsque β vaut zéro ?

`C ← αAB + βC` 中，α 与 β 分别起什么作用？β 为零时，对输入数据的需求发生什么变化?

Réponse proposée / 参考答案

α multiplie le produit matriciel AB, tandis que β pondère la valeur précédente de C. Mathématiquement, pour β=0, le résultat ne dépend plus du C initial. Une implémentation adaptée peut alors éviter de lire cet ancien C ; si le code évalue encore `β*C` , il faut néanmoins lui fournir une valeur initialisée valide, ou supprimer cette lecture.

α 缩放矩阵乘积 AB，β 缩放 C 的旧值。数学上，当 β=0 时，结果不再依赖 C 的初值。对应实现可以省去读取旧 C；如果代码仍然求值 `β*C` ，则需要保证被读取值已有效初始化，或直接去掉这次读取。

Repères / 复习讲义：4.7。

Q084

Prioritaire / 核心

Pourquoi comparer un GEMM GPU à une référence CPU avec une tolérance et des entrées variées, plutôt que d'exiger seulement une égalité exacte sur un cas simple ?

为什么校验 GPU GEMM 时，应结合容差和多种输入与 CPU 参考比较，而不能只看一个简单输入是否完全相等?

Réponse proposée / 参考答案

L'ordre des opérations et les instructions employées peuvent modifier légèrement l'arrondi en virgule flottante. Une tolérance absolue et relative adaptée permet d'interpréter ces petites différences, sans masquer de grandes erreurs. Des entrées variées, notamment un C initial non nul et des tailles non multiples du bloc, aident à révéler les erreurs de formule et d'indices.

浮点运算顺序和指令选择可能造成细小的舍入差异，因此应结合合适的绝对、相对容差判断。容差应能接纳合理舍入差异，同时不能掩盖明显错误。测试非零 C 初值、非整块尺寸等多种输入，更容易发现公式和下标错误。

Repères / 复习讲义：4.8。

Q085

Prioritaire / 核心

D’où viennent la complexité en N^3 et le compte usuel de $2N^3$ FLOP pour un GEMM de taille N×N ?

N×N 的 GEMM 为什么是 N^3 复杂度，常见的 $2N^3$ FLOP 计数又是如何得到的?

Réponse proposée / 参考答案

Il faut calculer N^2 éléments de sortie, chacun à partir d’une somme de N produits. Cela donne un travail proportionnel à N^3. En comptant une multiplication et une addition par étape, ou deux FLOP pour une FMA, le terme dominant vaut $2N^3$; les opérations finales liées à α et β ajoutent seulement un terme en N^2.

输出共有 N^2 个元素，每个元素都要累计 N 个乘积，因此总工作量与 N^3 成正比。若每轮按一次乘法和一次加法计数，或将一次 FMA 计为两个 FLOP，主项就是 $2N^3$ 。最后处理 α、β 的运算只增加 N^2 级别的项。

Repères / 复习讲义：4.8。

Q086

Prioritaire / 核心

Pourquoi faut-il annoncer le périmètre de chronométrage lorsqu’on donne un speedup CPU/GPU ?

报告 CPU/GPU 加速比时，为什么必须说明计时范围?

Réponse proposée / 参考答案

Le speedup est le rapport `t_CPU/t_GPU` , mais le temps GPU peut désigner le kernel seul ou inclure des copies, des allocations et d’autres opérations. Ces choix produisent des rapports différents. Il faut donc décrire les deux mesures et leurs périmètres pour que la comparaison soit interprétable.

加速比等于 `t_CPU/t_GPU` ，但 GPU 时间可能只计 kernel，也可能包含传输、分配等操作。不同范围会得到不同的加速比。因此必须说明 CPU 与 GPU 各自测了哪些步骤，才能解释比较结果。

Repères / 复习讲义：4.8。

Q087

Prioritaire / 核心

Comment passer d’un nombre d’opérations F et d’un temps t exprimé en millisecondes à un débit en GFLOP/s ?

已知运算量 F 和以毫秒表示的耗时 t，怎样计算 GFLOP/s?

Réponse proposée / 参考答案

Il faut d'abord convertir les millisecondes en secondes, puis diviser le nombre de FLOP par le temps. Comme un GFLOP vaut 10^9 FLOP, le résultat est $F/(t_ms*10^6)$ GFLOP/s. Les copies mémoire ne sont pas des FLOP ; si leur temps est inclus, on rapporte un débit effectif sur ce périmètre élargi.

先将毫秒转换为秒，再用 FLOP 数除以耗时。因为一个 GFLOP 等于 10^9 FLOP，公式可写为 $F/(t_ms*10^6)$ GFLOP/s。内存复制不计入 FLOP；若时间包含复制，得到的是该完整计时范围的有效吞吐率。

Repères / 复习讲义：4.8。

Q088

Prioritaire / 核心

Pourquoi la proportion de points vérifiant $x^2+y^2\le1$, tirés uniformément dans $[0,1]\times[0,1]$, permet-elle d'estimer π ?

在 $[0,1]\times[0,1]$ 中均匀取点，为什么统计满足 $x^2+y^2\le1$ 的点的比例，可以估计 π?

Réponse proposée / 参考答案

Le carré a une aire de 1 et la région retenue est un quart de disque de rayon 1, d'aire π/4. Pour des tirages uniformes, la proportion H/T des points dans ce quart de disque estime donc π/4. On obtient l'estimation $\pi \approx 4H/T$, avec une erreur statistique qui ne disparaît pas pour un nombre fini de tirs.

单位正方形面积为 1，满足条件的区域是半径为 1 的四分之一圆盘，面积为 π/4。均匀采样时，命中比例 H/T 就是对 π/4 的估计。因此可以用 $\pi \approx 4H/T$ ，有限次采样仍会存在统计误差。

Repères / 复习讲义：4.4。

Q089

Prioritaire / 核心

Pourquoi le TD2 peut-il moyenner les estimations de π produites par les threads ? Que faut-il changer si les nombres de tirs par thread diffèrent ?

为什么 TD2 可以对各线程的 π 估计值直接求平均？如果各线程采样次数不同，需要怎样调整?

Réponse proposée / 参考答案

Dans le sujet, chaque thread effectue le même nombre de tirs : la moyenne simple de leurs estimations équivaut donc à regrouper tous les tirs. Si les nombres diffèrent, une moyenne simple

donne trop de poids aux petits échantillons. Il faut pondérer par le nombre de tirs, ou sommer les succès et les tirs puis calculer `4*sum(hits)/sum(trials)` .

原题中各线程采样次数相同，直接平均估计值等价于汇总全部采样。若采样次数不同，简单平均会让采样较少的线程获得过高权重。此时应按采样次数加权，或者先求总命中数和总采样数，再计算 `4*sum(hits)/sum(trials)` 。

Repères / 复习讲义：4.5。

Q090

Complément / 补充

Pourquoi le kernel Monte Carlo utilise-t-il un état pseudo-aléatoire local et une séquence distinguée par l’identifiant du thread ?

Monte Carlo kernel 为什么为每个线程使用局部伪随机数状态，并用线程编号区分序列?

Réponse proposée / 参考答案

Un état local évite que plusieurs threads modifient concurremment le même état du générateur. Des états initialisés exactement de la même façon et utilisés pareil produiraient des suites identiques. Distinguer les séquences limite cette duplication, mais la qualité statistique dépend toujours du générateur et de sa configuration.

局部状态避免多个线程并发修改同一个生成器状态。若初始状态完全相同，调用方式也相同，各线程会生成相同序列。区分序列可以避免这种重复，但统计质量仍取决于生成器及其配置。

Repères / 复习讲义：4.5。

Q091

Prioritaire / 核心

Pourquoi choisir la Driver API plutôt que la Runtime API ne garantit-il pas automatiquement une meilleure performance ?

为什么选择 Driver API，而不是 Runtime API，并不能自动保证性能更好?

Réponse proposée / 参考答案

Les deux API permettent d’utiliser le même matériel GPU. La Driver API donne un contrôle plus explicite, notamment sur les contextes et certaines ressources, tandis que la Runtime API simplifie leur gestion. Un gain de performance dépend de l’usage concret de ces possibilités et du travail exécuté, pas du seul nom de l’API.

两套 API 都可以使用同一块 GPU。Driver API 提供更显式的上下文和资源控制，Runtime API 则简化这些管理。是否更快取决于如何运用这些控制以及实际工作负载，而不是 API 的名字。

Repères / 复习讲义：3.1。

Q092

Prioritaire / 核心

À quoi sert un contexte CUDA et pourquoi est-il utile de connaître le contexte associé à une ressource ?

CUDA 上下文有什么作用？为什么需要知道某项资源属于哪个上下文？

Réponse proposée / 参考答案

Un contexte organise un état CUDA lié à un GPU, avec des ressources telles que les allocations, les streams et les événements. Connaître cette appartenance permet de vérifier que les opérations utilisent des ressources compatibles avec le contexte courant. Le seul numéro de GPU ne décrit pas toutes ces relations.

上下文组织某个 GPU 的 CUDA 状态，其中包括内存分配、stream 和 event 等资源。知道资源归属，才能检查当前操作使用的资源是否与当前上下文相容。仅知道 GPU 编号，无法表达全部这些关系。

Repères / 复习讲义：3.2。

Q093

Complément / 补充

Que faut-il vérifier lorsqu'un thread hôte veut lancer un calcul sur le GPU 1 après avoir travaillé sur le GPU 0 ?

某个主机线程原先在 GPU 0 上工作，现在希望在 GPU 1 上启动计算，需要检查什么？

Réponse proposée / 参考答案

Il faut sélectionner le GPU voulu pour le thread hôte et vérifier que les buffers et autres ressources utilisés conviennent à ce GPU. `cudaSetDevice` ne déplace pas les données précédemment allouées. Si le nouveau calcul dépend de données produites ailleurs, leur disponibilité et l'ordre des opérations doivent être assurés séparément.

需要为该主机线程选择目标 GPU，并检查 buffer 等资源能否在该 GPU 上使用。 `cudaSetDevice` 不会迁移此前分配的数据。若新计算依赖其他设备产生的数据，还需要另行保证数据可用和操作顺序正确。

Repères / 复习讲义：3.3。

Q094

Complément / 补充

Pourquoi faut-il utiliser le nombre de GPU visibles par le processus au lieu de supposer les numéros des cartes physiques de la machine ?

为什么应根据进程可见的 GPU 数量选择设备，而不能直接假设整台机器的物理显卡编号都可用?

Réponse proposée / 参考答案

L'environnement peut restreindre les GPU accessibles au processus et présenter une numérotation locale. `cudaGetDeviceCount` décrit les périphériques visibles dans cet environnement. Il faut vérifier son résultat et utiliser un identifiant valide ; un identifiant invalide ne doit pas être supposé redirigé automatiquement vers le GPU 0.

运行环境可能限制进程可访问的 GPU，并使用该环境中的本地编号。`cudaGetDeviceCount` 返回的是这里可见的设备数。应检查返回结果并选择有效编号，不能假定非法编号会自动改为 GPU 0。

Repères / 复习讲义：3.3。

Q095

Prioritaire / 核心

Expliquez la différence entre unification des adresses avec UVA et gestion des données avec la mémoire unifiée.

请解释 UVA 的统一地址与统一内存的数据管理有什么区别。

Réponse proposée / 参考答案

UVA fournit une organisation unifiée des adresses virtuelles de l'hôte et des GPU pris en charge. Une allocation managed fournit une mémoire accessible au CPU et au GPU, avec une gestion par le système des accès et des mouvements nécessaires. UVA seul ne transforme pas chaque allocation en mémoire managed et ne supprime ni les restrictions d'accès ni les dépendances de données.

UVA 为受支持平台上的主机、GPU 提供统一的虚拟地址布局。managed 分配则提供 CPU/GPU 可访问的内存，并由系统管理所需的访问与数据移动。仅有 UVA 不会让所有分配变成 managed 内存，也不会消除访问限制和数据依赖。

Repères / 复习讲义：3.5。

Q096

Complément / 补充

Dans un environnement UVA, pourquoi « le système identifie ce pointeur » ne signifie-t-il pas « le CPU peut le déréférencer » ?

UVA 环境中，为什么“系统能识别这个指针”不等于“CPU 可以解引用这个指针”?

Réponse proposée / 参考答案

L'identification d'une adresse permet de la relier à une allocation, mais les possibilités d'accès dépendent de la nature et de l'emplacement de cette allocation. Par exemple, une allocation ordinaire en mémoire device n'est pas rendue directement lisible par le CPU du seul fait de la présence d'UVA. Il faut examiner séparément la localisation, les droits d'accès et la disponibilité des données.

地址识别可以将地址对应到某项分配，但能否访问仍取决于分配类型和所在位置。例如，普通 device 内存不会仅因为支持 UVA，就自动变成 CPU 可直接读取的内存。应分别判断数据位置、访问权限和数据是否就绪。

Repères / 复习讲义：3.5。

Q097

Complément / 补充

Pourquoi deux ressources créées sur le même GPU physique ne sont-elles pas nécessairement utilisables ensemble sans autre vérification ?

为什么两项资源即使建立在同一个物理 GPU 上，也不一定能在未经检查时直接混用?

Réponse proposée / 参考答案

Le GPU physique et le contexte sont deux notions distinctes. Deux contextes explicites sur le même GPU peuvent organiser des espaces et des ressources distincts. Il faut donc vérifier le contexte courant et l'appartenance des objets, plutôt que de conclure à leur compatibilité à partir du seul numéro de GPU.

物理 GPU 与上下文是两个不同概念。同一 GPU 上的两个显式上下文可以组织不同的地址空间和资源。因此需要检查当前上下文和对象归属，不能只看 GPU 编号就认定资源兼容。

Repères / 复习讲义：3.2。

Q098

Complément / 补充

Quel rôle jouent `target`, `teams`, `distribute` et `parallel for` dans l'exemple OpenMP d'addition vectorielle ?

OpenMP 向量加法示例中的 `target`、`teams`、`distribute`、`parallel for` 分别承担什么职责?

Réponse proposée / 参考答案

`target` définit la région destinée au dispositif cible et `teams` établit des équipes. `distribute` répartit les itérations entre les équipes, puis `parallel for` les partage entre les threads d'une équipe. Cette description ne garantit pas une correspondance portable exacte entre équipe OpenMP et bloc CUDA.

`target` 定义交给目标设备执行的区域，`teams` 建立线程团队。`distribute` 在团队之间分配迭代，`parallel for` 再将迭代分配给团队内的线程。不能把 OpenMP 团队与 CUDA block 的一一对应当成可移植保证。

Repères / 复习讲义：3.13。

Q099

Complément / 补充

Une région target s'exécute effectivement sur un dispositif dont la mémoire est distincte de celle de l'hôte, sans mapping préexistant. Pourquoi peut-on utiliser `map(from: c[0:N])` pour `c=a+b` , mais faut-il `map(tofrom: y[0:N])` pour `y=alpha*x+y` , si l'hôte fournit les entrées et récupère les résultats ?

一个 target 区域实际在内存独立于主机的设备上执行，不存在既有映射，输入由主机提供，结果需要取回主机。为什么 `c=a+b` 的 c 可以用 `map(from: c[0:N])` ，而 `y=alpha*x+y` 的 y 需要 `map(tofrom: y[0:N])` ？

Réponse proposée / 参考答案

L'addition `c=a+b` écrase c sans lire son ancienne valeur : il suffit de récupérer la sortie. L'AXPY lit la valeur initiale de y puis la modifie ; celle-ci doit être disponible sur le dispositif et le résultat doit revenir à l'hôte. `tofrom` exprime ces deux besoins, et `[0:N]` désigne N éléments à partir de l'indice zéro.

`c=a+b` 会覆盖 c，计算不读取 c 的旧值，所以只需要取回输出。AXPY 先读取 y 初值再更新，因此既需要将初值提供给设备，也需要将结果带回主机。`tofrom` 表达这两个方向，`[0:N]` 表示从下标 0 开始的 N 个元素。

Repères / 复习讲义：3.13。

Q100

Complément / 补充

Une région OpenMP target donne le résultat attendu. Cela suffit-il à prouver qu'elle a été exécutée sur le GPU ?

OpenMP target 区域输出了正确结果，这是否足以证明它在 GPU 上执行?

Réponse proposée / 参考答案

Non : l'implémentation peut exécuter la région sur l'hôte si le dispositif cible n'est pas utilisé. Il faut vérifier la configuration de compilation et le lieu d'exécution ; dans l'exemple du cours, `omp_is_initial_device` appelé dans la région permet d'identifier un repli sur l'hôte. Un résultat nul indique un dispositif autre que le dispositif initial, dont le type doit être connu par la configuration.

不能，因为目标设备没有实际使用时，实现可能在主机上执行该区域。需要检查编译配置和实际执行位置；课堂示例在区域中调用 `omp_is_initial_device` 来识别是否在主机上执行。返回零只说明当前设备不是初始设备，具体设备类型还要结合配置判断。

Repères / 复习讲义：3.14。

Revenir au parcours prioritaire / 返回核心题索引